LTE State Machine Test Suite

Reference report

What every test in the LTE test harness does, how the harness simulates the modem, and which part of `lte_sm.c` each test protects.

Covers `lte_sm.c` and the test file (`lte_run_tests`)
Tests 30 test runs, grouped in 8 areas
Date 3 November 2026

Note

This report was written by reading the source code of `lte_sm.c` and of the test file. The suite was *not* compiled or run to produce it, so every "expected" result is what the code should do according to the source. If a real run disagrees, trust the run and look for the reason.

Contents

1 About this report **3**

2 The code under test **3**

2.1 The core rule . . . 3
2.2 The happy path . . . 3
2.3 What each state sends, and where failures go . . . 4
2.4 Constants the tests depend on . . . 4
2.5 The boot sequence . . . 4
2.6 Failure handling rules . . . 5
2.7 How one line from the modem is routed . . . 5
2.8 How `+CEREG` reports are treated . . . 5

3 The test harness **6**

3.1 The principle . . . 6
3.2 Helper functions . . . 6
3.3 Why there are two ways to run the machine . . . 6
3.4 Start helpers . . . 6
3.5 Reusable test functions . . . 7
3.6 The runner . . . 7

4 Test catalogue **7**

Group 1: happy path . . . 7
4.1 `test_attach_ok` . . . 7
4.2 `test_pdp_ok` . . . 8
4.3 `test_full_path_to_run` . . . 8
Group 2: failure replies . . . 8
4.4 `test_attach_error_retry` . . . 8
4.5 `test_reply_recovery (four runs)` . . . 9
4.6 `test_cme_error_is_failure` . . . 9
Group 3: timeouts . . . 9
4.7 `test_timeout_attach` . . . 9
4.8 `test_timeout_recovery (four runs)` . . . 10
Group 4: retry limit and reboot . . . 10
4.9 `test_retries_exhausted_reboot` . . . 10
4.10 `test_retries_exhausted_for (four runs)` . . . 11
Group 5: retry bookkeeping . . . 11

4.11 `test_attach_fail_once_then_ok` . . . 11
4.12 `test_retry_counter_reset` . . . 11
Group 6: boot sequence . . . 11
4.13 `test_boot_silence_reboot` . . . 12
4.14 `test_sim_not_ready` . . . 12
4.15 `test_boot_noncritical_errors` . . . 12
Group 7: registration reports . . . 12
4.16 `test_cereg_network_lost` . . . 12
4.17 `test_cereg_lost_before_attach_ignored` . . . 13
4.18 `test_cereg_registered_no_change` . . . 13
4.19 `test_cereg_denied` . . . 13
Group 8: running state . . . 13
4.20 `test_failure_in_run_reboots` . . . 13
4.21 `test_stray_ok_in_run` . . . 14

5 Quick lookup table **14**

6 Key ideas **15**
6.1 Interview sentences . . . 15

7 Limits of the suite **15**
7.1 What the tests do not check . . . 15
7.2 Behaviour with no test . . . 16
7.3 Behaviour the code does not have yet . . . 16
7.4 Candidate additions . . . 16

A Glossary **17**

1 About this report

The test file simulates a modem. It never talks to hardware: it feeds fake replies to the state machine and checks which state the machine ends in. This report explains that idea first (sections 2 and 3), then describes every test one by one (section 4), and finally gives a lookup table, the key ideas, and the limits of the suite.

Conventions.

- State names are written without the `LTE_SM_` prefix, so `ATTACH` means `LTE_SM_ATTACH`.
- "Tick" means `lte_sm_handle_event(NULL)`: the periodic call from the main loop that lets the machine send the next command or run recovery.
- Each test is described on a card with five rows: Purpose, Setup, Action, Expected, and Why it matters.
- Times are in milliseconds (ms) and run on a fake clock.

2 The code under test

2.1 The core rule

AT replies carry no identifier, so an `OK` does not say which command it answers. The state machine therefore follows one rule everywhere:

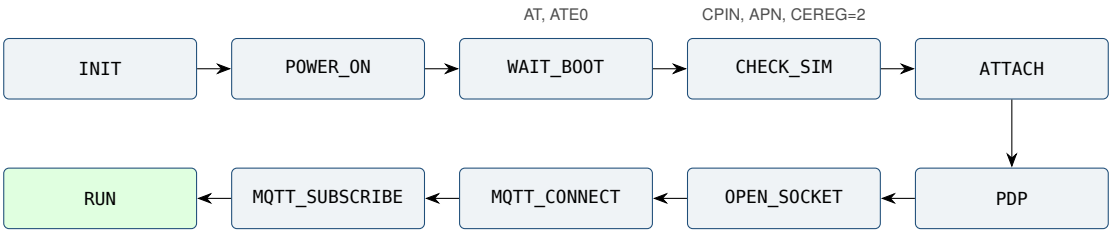
Key idea

One command in flight at a time. Send a command, wait for its final result (`OK` or an error), then send the next one. The variable `s_current_cmd` holds the command that is in flight.

Three kinds of line come from the modem:

- **Final result:** `OK`, `ERROR`, `+CME ERROR: 30`. It ends a command.
- **Information line:** for example `+CPIN: READY`. It arrives *before* the final `OK` and does not end the command.
- **URC** (unsolicited result code): for example `+CEREG: 1`. The modem may send it at any time. It belongs to no command.

2.2 The happy path



`WAIT_BOOT` and `CHECK_SIM` each contain small sub-steps (section 2.5), so no extra states were needed.

2.3 What each state sends, and where failures go

State	Command sent	On OK	On error or timeout
ATTACH	AT+CGATT=1	PDP	REATTACH
PDP	AT+CGACT=1,1	OPEN_SOCKET	REPDP
OPEN_SOCKET	lte_tcp_open()	MQTT_CONNECT	REOPEN_SOCKET
MQTT_CONNECT	lte_mqtt_connect()	MQTT_SUBSCRIBE	MQTT_RECONNECT
MQTT_SUBSCRIBE	lte_mqtt_subscribe()	RUN	MQTT_RESUBSCRIBE

Each RE... state is a recovery state. On the next tick it resends the command and returns to the normal state to wait for the result.

2.4 Constants the tests depend on

Constant	Value	Meaning
MAX_RETRIES	3	Failures of one command before the modem is rebooted
TIMEOUT_ATTACH	30000 ms	Attach is long: a weak network may search for a cell
TIMEOUT_PDP	10000 ms	PDP activation
TIMEOUT_TCP_OPEN	8000 ms	TCP socket open
TIMEOUT_MQTT_CONNECT	8000 ms	MQTT connect
TIMEOUT_MQTT_SUB	8000 ms	MQTT subscribe
BOOT_STEP_TIMEOUT_MS	2000 ms	Wait for a boot reply before asking again
BOOT_MAX_SENDS	15	Sends of one boot command before rebooting

The timeout watchdog fires when the elapsed time is *greater than* the timeout, not equal to it. That is why the timeout tests advance the clock by `timeout + 1` ms.

2.5 The boot sequence

Boot commands are not tracked by `s_current_cmd`. They have their own step variable and their own send counter (`s_boot_sends`). Every boot command waits for its own reply before the next is sent.

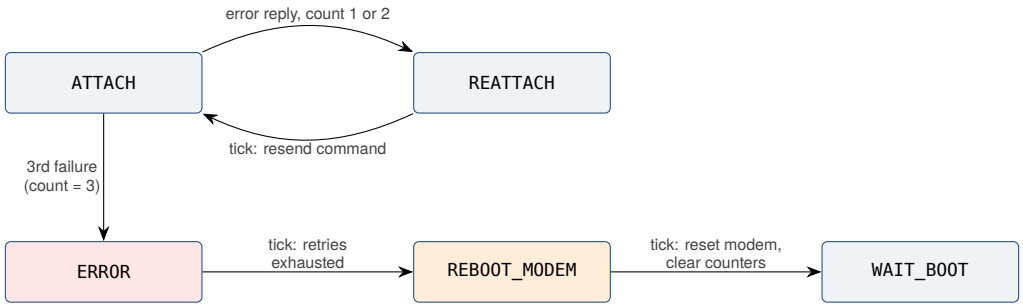
Step	Command	State	Moves on when
AT	AT	WAIT_BOOT	OK
ATE0	ATE0	WAIT_BOOT	OK
CPIN	AT+CPIN?	CHECK_SIM	+CPIN: READY and then OK
APN	AT+CGDCONT=\dots	CHECK_SIM	OK, or an error (not critical)
CEREG	AT+CEREG=2	CHECK_SIM	OK, or an error (not critical)
DONE	AT+CGATT=1	ATTACH	attach is started

If no usable reply comes within 2000 ms, the same command is sent again. After 15 sends of one step the machine goes to ERROR, and because a boot failure has no specific recovery state, it then reboots the modem. The send counter restarts at 0 for every new step.

2.6 Failure handling rules

1. A **failure** is an error reply (`ERROR`, `+CME ERROR`, `+CMS ERROR`) or a timeout. It adds 1 to that command's retry counter.
2. A **success** resets that command's counter to 0.
3. Counter below 3: go to the `RE...` state; the next tick resends the command.
4. Counter reaches 3: go to `ERROR`; the next tick sees "retries exhausted" and chooses `REBOOT_MODEM`; the tick after that resets the modem, clears all counters, restarts the boot sequence, and enters `WAIT_BOOT`.
5. A failure in a state with no specific recovery (boot states, `RUN`) goes straight to `REBOOT_MODEM` through `ERROR`.

A timeout always enters `ERROR` first and is then mapped to the recovery state on the same tick. An error reply below the limit goes to the recovery state directly.



2.7 How one line from the modem is routed

`lte_sm_handle_event` first runs the timeout check, then routes a received line in this order:

A. A `+CEREG:` line goes to the registration (URC) handler. It never belongs to a command.

B. If a command is in flight, that command's handler gets the line.

C. If the state is `ERROR`, choose a recovery state.

D. If the state is a recovery state, run the recovery action.

E. Otherwise the forward path handles it (boot steps use the line here).

This order is why the tests must put a command in flight before injecting a reply: without it, the reply would fall through to steps C–E.

2.8 How `+CEREG` reports are treated

`stat`	Meaning	What the machine does
0	not registered	If the network is expected, go to `REATTACH`; otherwise ignore
1, 5	registered (home, roaming)	Nothing
2, 4	searching, unknown	Nothing
3	registration denied	Add 1 to the attach counter and go to `ERROR`

The network is "expected" in `PDP`, `OPEN_SOCKET`, `MQTT_CONNECT`, `MQTT_SUBSCRIBE`, `RUN`, and their recovery states. It is not expected in `ATTACH` or the boot states, where "not registered" is normal. The parser accepts both forms: the query form `+CEREG: 2,1` and the URC form `+CEREG: 1`.

3 The test harness

3.1 The principle

Every test follows the same three steps:

1. **Arrange:** put the state machine in a known state.
2. **Act:** inject a fake modem reply, or move the fake clock forward.
3. **Assert:** check that the state is the expected one.

3.2 Helper functions

Helper	What it does
`inject_line(line)`	Sends the characters of the line, plus carriage return and line feed, into the driver as if they arrived from the UART. The driver turns them into one line event.
`step_sm()`	One pass of the main loop: handle every waiting event, then run one tick.
`deliver_line(line)`	Inject a line and handle *only* that event, with no tick afterwards.
`modem_reply(line)`	`deliver_line` followed by `step_sm`: what the real main loop does.
`advance_time_ms(ms)`	Move the fake clock 1 ms at a time, running `step_sm` after each millisecond.
`check_state(expect, what)`	Compare the current state with the expected one and print PASS or FAIL.

The fake clock only moves when `lte_hw_delay_ms()` is called. A 30-second timeout therefore takes almost no real time.

3.3 Why there are two ways to run the machine

A tick makes the machine recover *immediately*. Consider an attach failure:

- With `deliver_line("ERROR")` only, the machine is in `REATTACH`. The test can see it.
- With `step_sm()` added, the tick resends the command and the state is already back in `ATTACH`. The state the test wanted to check has gone.

So tests that check a state *right after a failure* use `deliver_line`. Tests that need the next command to be sent use `step_sm` or `modem_reply`.

3.4 Start helpers

The start helpers skip the long bring-up and put the machine in a known state with a command already in flight, so that an injected reply reaches the right handler.

Helper	Forced state	After one tick	Command in flight
`sm_start_in_attach`	`REATTACH`	`ATTACH`	`AT+CGATT=1`
`sm_start_in_pdp`	`REPDP`	`PDP`	`AT+CGACT=1,1`
`sm_start_in_socket`	`REOPEN_SOCKET`	`OPEN_SOCKET`	TCP open
`sm_start_in_mqtt_connect`	`MQTT_RECONNECT`	`MQTT_CONNECT`	MQTT connect
`sm_start_in_mqtt_subscribe`	`MQTT_RESUBSCRIBE`	`MQTT_SUBSCRIBE`	MQTT subscribe
`sm_start_boot`	(power on, two ticks)	`WAIT_BOOT`	`AT` (boot step)

3.5 Reusable test functions

Three functions take a start helper as a parameter (`start_fn_t`), so one function can test PDP, TCP, MQTT connect, and MQTT subscribe instead of four copies of the same code.

- `test_reply_recovery(name, start, reply, expect)`: start, deliver one reply, check the state.
- `test_timeout_recovery(name, start, timeout_ms, expect)`: start, advance `timeout + 1` ms, check the state.
- `test_retries_exhausted_for(name, start, recovery)`: fail the command three times and follow the escalation to `WAIT_BOOT`.

3.6 The runner

`lte_run_tests()` calls every test, counts passes, prints `TEST SUMMARY: pass/total`, and returns the number of *failures* (0 means all passed). `lte_run_all_tests()` wraps it for callers such as the ESP32 application. `lte_hw_init()` is called first to prepare the hardware layer in test mode.

4 Test catalogue

Group 1: happy path

4.1 `test_attach_ok`

Purpose	A successful attach moves the machine to `PDP`.
Setup	`sm_start_in_attach()`: state `ATTACH`, `AT+CGATT=1` in flight.
Action	Inject `+CGATT:1` and tick; inject `OK` and tick.
Expected	State is `PDP`.
Why it matters	The `+CGATT:` line is an information line and must not change the state. Only the final `OK` moves on. The handler treats only the exact text `+CGATT: 0` as a failure.

4.2 `test_pdp_ok`

Purpose	A successful PDP activation moves the machine to `OPEN_SOCKET`.
Setup	`sm_start_in_pdp()`: state `PDP`, `AT+CGACT=1,1` in flight.
Action	Inject `OK` and tick.
Expected	State is `OPEN_SOCKET`.
Why it matters	Proves the success path of the PDP handler and that the next tick opens the socket.

4.3 `test_full_path_to_run`

Purpose	The whole bring-up from `INIT` to `RUN`, one modem reply at a time.
Setup	`lte_sm_init()` and two ticks: `INIT` to `POWER_ON` to `WAIT_BOOT` (first `AT` sent).
Action	For each row of the table below: deliver the reply, tick, and compare the state.
Expected	State is `RUN` after the last reply, with every intermediate state matching its row.
Why it matters	The most important integration test. It shows that every command waits for its own reply and that no stray `OK` can skip a step.

#	Reply	State after	What the machine did
1	`OK`	`WAIT_BOOT`	answer to AT; sends `ATE0`
2	`OK`	`CHECK_SIM`	answer to ATE0; sends `AT+CPIN?`
3	`+CPIN: READY`	`CHECK_SIM`	information line; remembers SIM is ready
4	`OK`	`CHECK_SIM`	end of CPIN; sends the APN command
5	`OK`	`CHECK_SIM`	answer to APN; sends `AT+CEREG=2`
6	`OK`	`ATTACH`	answer to CEREG; sends `AT+CGATT=1`
7	`OK`	`PDP`	attach done; sends `AT+CGACT=1,1`
8	`OK`	`OPEN_SOCKET`	PDP active; opens TCP
9	`OK`	`MQTT_CONNECT`	TCP open; sends MQTT connect
10	`OK`	`MQTT_SUBSCRIBE`	MQTT connected; sends subscribe
11	`OK`	`RUN`	subscribed; running

Group 2: failure replies

4.4 `test_attach_error_retry`

Purpose	An `ERROR` reply to the attach leads to a recovery state.
Setup	`sm_start_in_attach()`.
Action	`deliver_line("ERROR")` with no tick.
Expected	State is `REATTACH` or `ERROR` (the test accepts both).
Why it matters	With the current code the counter is 1, so the real result is `REATTACH`. Accepting `ERROR` as well makes the test looser than it needs to be (see section 7).

4.5 `test_reply_recovery (four runs)`

Purpose	Each command has its own recovery state.
Setup	The start helper of the command under test.
Action	`deliver_line("ERROR")`.
Expected	The recovery state in the table below.
Why it matters	Proves that every handler is wired to the right recovery state, not only the attach handler.

Run name	Start helper	Reply	Expected state
PDP error	`sm_start_in_pdp`	`ERROR`	`REPDP`
TCP open error	`sm_start_in_socket`	`ERROR`	`REOPEN_SOCKET`
MQTT connect error	`sm_start_in_mqtt_connect`	`ERROR`	`MQTT_RECONNECT`
MQTT subscribe error	`sm_start_in_mqtt_subscribe`	`ERROR`	`MQTT_RESUBSCRIBE`

4.6 `test_cme_error_is_failure`

Purpose	A `+CME ERROR` is recognised as a failure, like a plain `ERROR`.
Setup	`sm_start_in_attach()`.
Action	`deliver_line("+CME ERROR: 30")` (through `test_reply_recovery`).
Expected	State is `REATTACH`.
Why it matters	Modems often answer `+CME ERROR: <code>` instead of a bare `ERROR`. The recogniser matches the prefix, so the code number does not matter.

Group 3: timeouts (the modem stays silent)

4.7 `test_timeout_attach`

Purpose	Silence for longer than the attach timeout counts as a failure.
Setup	`sm_start_in_attach()`: `AT+CGATT=1` sent, no reply will come.
Action	`advance_time_ms(30001)`.
Expected	State is `ERROR` or `REATTACH` (the test accepts both).
Why it matters	The watchdog fires when more than 30000 ms have passed, hence 30001. On the same tick the machine maps `ERROR` to `REATTACH`, so that is what is normally seen. Advancing further would let the machine resend the command and start a new wait.

4.8 `test_timeout_recovery (four runs)`

Purpose	Each command has its own timeout and its own recovery state.
Setup	The start helper of the command under test.
Action	`advance_time_ms(timeout + 1)` with no reply.
Expected	The recovery state in the table below.
Why it matters	A silent modem sends no line, so only the tick-driven timeout can detect the failure.

Run name	Timeout	Clock advance	Expected state
PDP timeout	10000 ms	10001 ms	`REPDP`
TCP open timeout	8000 ms	8001 ms	`REOPEN_SOCKET`
MQTT connect timeout	8000 ms	8001 ms	`MQTT_RECONNECT`
MQTT subscribe timeout	8000 ms	8001 ms	`MQTT_RESUBSCRIBE`

Group 4: retry limit and reboot

4.9 `test_retries_exhausted_reboot`

Purpose	Three failures of the same command stop the retrying and reboot the modem.
Setup	`sm_start_in_attach(); retry_attach = 0`.
Action	Three times `deliver_line("ERROR")`, with a tick after the first and second to resend. Then two more ticks.
Expected	`REATTACH` after failure 1 and 2; `ERROR` after failure 3; then `REBOOT_MODEM`; then `WAIT_BOOT`.
Why it matters	Proves the escalation: bounded retries, then a heavier recovery. Without it, a dead modem would loop between `ERROR` and `REATTACH` forever.

Pass	Test does	Machine does	State
1	`deliver_line("ERROR")`	counter 1, below limit	`REATTACH`
1	`step_sm()`	tick resends the command	`ATTACH`
2	`deliver_line("ERROR")`	counter 2, below limit	`REATTACH`
2	`step_sm()`	tick resends the command	`ATTACH`
3	`deliver_line("ERROR")`	counter 3 = `MAX_RETRIES`	`ERROR`
–	`step_sm()`	retries exhausted, choose reboot	`REBOOT_MODEM`
–	`step_sm()`	reset modem, clear counters, send `AT`	`WAIT_BOOT`

The command is sent three times in total (the first send plus two resends); the third failure triggers the reboot. The value `max_retries = 3` is written into the test and must match `MAX_RETRIES` in `lte_sm.c`.

4.10 `test_retries_exhausted_for (four runs)`

Purpose	The same escalation as above for the other four commands.
Setup	The start helper of the command under test.
Action	Three times `ERROR`, ticking after failures 1 and 2; then two more ticks.
Expected	Recovery state after failures 1 and 2; then `ERROR`, `REBOOT_MODEM`, `WAIT_BOOT`.
Why it matters	Each command has its own counter, so each needs its own proof that it escalates.

Run name	Start helper	Recovery state checked
PDP retries exhausted	`sm_start_in_pdp`	`REPDP`
TCP retries exhausted	`sm_start_in_socket`	`REOPEN_SOCKET`
MQTT connect retries exhausted	`sm_start_in_mqtt_connect`	`MQTT_RECONNECT`
MQTT subscribe retries exhausted	`sm_start_in_mqtt_subscribe`	`MQTT_RESUBSCRIBE`

Group 5: retry bookkeeping

4.11 `test_attach_fail_once_then_ok`

Purpose	One failed attach followed by a successful retry.
Setup	`sm_start_in_attach()`.
Action	`modem_reply("ERROR")`, check the state, then `deliver_line("OK")`.
Expected	`ATTACH` after the failure (command resent), then `PDP`.
Why it matters	Proves the retry really resends the command and that the retry can succeed.

4.12 `test_retry_counter_reset`

Purpose	A success clears the retry counter, so old failures do not add up.
Setup	`sm_start_in_attach()`.
Action	Two failures (`modem_reply("ERROR")` twice), then `OK`, then force `REATTACH`, tick, and deliver one more `ERROR`.
Expected	`PDP` after the `OK`; then `REATTACH` (not `ERROR`) after the new failure.
Why it matters	If the counter were not cleared, the new failure would be number 3 and would trigger a reboot. Only *consecutive* failures count.

Group 6: boot sequence

4.13 test_boot_silence_reboot

Purpose	A modem that never answers AT is asked again and again, then rebooted.
Setup	lte_sm_init() and two ticks: state WAIT_BOOT, first AT sent.
Action	Never inject a reply. Move the clock 1 ms at a time, ticking, until the state leaves WAIT_BOOT (capped at 40000 ms so a bug cannot hang the test).
Expected	ERROR after about 30000 ms (accepted between 28000 and 32000); then REBOOT_MODEM; then WAIT_BOOT.
Why it matters	Boot has its own retry logic: 15 sends, 2000 ms apart ($15 \times 2000 = 30000$ ms). The retry counters are all 0 here. The reboot comes from the default case of the recovery mapping: a failure in WAIT_BOOT has no specific recovery state.

4.14 test_sim_not_ready

Purpose	A locked SIM blocks the boot, and the boot continues once the SIM is ready.
Setup	sm_start_boot(), then OK for AT and OK for ATE0; AT+CPIN? is now in flight.
Action	Reply +CPIN: SIM PIN then three OK; check the state; advance 2000 ms; reply +CPIN: READY then three OK.
Expected	CHECK_SIM while the SIM is locked (boot must wait); ATTACH after the SIM is ready.
Why it matters	An OK without READY must not advance the boot. After the step timeout the machine asks again, and only then can the boot go on. This is the guard against a stray OK.

4.15 test_boot_noncritical_errors

Purpose	Errors on the APN and CEREG steps do not stop the boot.
Setup	sm_start_boot(); OK for AT and ATE0; +CPIN: READY; OK. The APN command is in flight.
Action	Reply ERROR to the APN command and ERROR to the CEREG command.
Expected	State is ATTACH.
Why it matters	Those two settings are helpful but not required to attempt an attach, so a failure is logged and the boot carries on.

Group 7: registration reports (+CEREG)

4.16 test_cereg_network_lost

Purpose	+CEREG: 0,0 after the network was up means the network is lost.
Setup	sm_start_in_pdp(): a state where the network is expected.
Action	deliver_line("+CEREG: 0,0").
Expected	State is ERROR or REATTACH (the test accepts both).
Why it matters	The URC handler drops the active command and goes straight to REATTACH. The query form 0,0 is parsed as stat = 0.

4.17 test_cereg_lost_before_attach_ignored

Purpose	"Not registered" is normal while attaching and must be ignored.
Setup	`sm_start_in_attach()`.
Action	`deliver_line("+CEREG: 0,0")`.
Expected	State is still `ATTACH`.
Why it matters	The attach command's own timeout covers real failures. Reacting here would make the machine fight its own attach.

4.18 test_cereg_registered_no_change

Purpose	`+CEREG: 1` (registered) changes nothing.
Setup	`sm_start_in_pdp()`.
Action	`deliver_line("+CEREG: 1")`.
Expected	State is still `PDP`.
Why it matters	A good report must not disturb a command that is in flight. It also exercises the URC form of the parser.

4.19 test_cereg_denied

Purpose	Registration denied is a failure that leads to a re-attach.
Setup	`sm_start_in_attach()`.
Action	`deliver_line("+CEREG: 3")`, check the state, then `step_sm()`.
Expected	`ERROR` first; then `REATTACH`.
Why it matters	The handler adds 1 to the attach counter before entering `ERROR`, so repeated denials count towards a reboot.

Group 8: running state

4.20 test_failure_in_run_reboots

Purpose	A failure while running reboots the modem.
Setup	`lte_sm_init()` and force `RUN`.
Action	Send an external `LTE_EVENT_ERROR` event, check the state, then `step_sm()`.
Expected	`ERROR`; then `REBOOT_MODEM`.
Why it matters	`RUN` has no specific recovery state, so the `default` branch of the recovery mapping chooses a reboot.

4.21 test_stray_ok_in_run

Purpose	A stray OK when nothing is pending does not disturb the machine.
Setup	lte_sm_init() and force RUN; no command in flight.
Action	deliver_line("OK").
Expected	State is still RUN.
Why it matters	AT replies carry no ID, so a late or duplicate OK must be harmless.

5 Quick lookup table

The tests in the order the runner executes them.

#	Test	Trigger	Final state
1	test_attach_ok	OK	PDP
2	test_attach_error_retry	ERROR	REATTACH or ERROR
3	test_pdp_ok	OK	OPEN_SOCKET
4	test_cereg_network_lost	+CEREG: 0,0	REATTACH or ERROR
5	test_timeout_attach	30001 ms silence	REATTACH or ERROR
6	test_full_path_to_run	11 replies	RUN
7	test_retries_exhausted_reboot	3 errors (attach)	WAIT_BOOT
8	test_boot_silence_reboot	30 s silence at boot	WAIT_BOOT
9	PDP error	ERROR	REPDP
10	TCP open error	ERROR	REOPEN_SOCKET
11	MQTT connect error	ERROR	MQTT_RECONNECT
12	MQTT subscribe error	ERROR	MQTT_RESUBSCRIBE
13	test_cme_error_is_failure	+CME ERROR: 30	REATTACH
14	PDP timeout	10001 ms silence	REPDP
15	TCP open timeout	8001 ms silence	REOPEN_SOCKET
16	MQTT connect timeout	8001 ms silence	MQTT_RECONNECT
17	MQTT subscribe timeout	8001 ms silence	MQTT_RESUBSCRIBE
18	PDP retries exhausted	3 errors	WAIT_BOOT
19	TCP retries exhausted	3 errors	WAIT_BOOT
20	MQTT connect retries exhausted	3 errors	WAIT_BOOT
21	MQTT subscribe retries exhausted	3 errors	WAIT_BOOT
22	test_attach_fail_once_then_ok	ERROR, OK	PDP
23	test_retry_counter_reset	2 errors, OK, error	REATTACH
24	test_sim_not_ready	+CPIN: SIM PIN	ATTACH
25	test_boot_noncritical_errors	errors on APN, CEREG	ATTACH
26	test_cereg_lost_before_attach_ignored	+CEREG: 0,0	ATTACH
27	test_cereg_registered_no_change	+CEREG: 1	PDP
28	test_cereg_denied	+CEREG: 3	REATTACH
29	test_failure_in_run_reboots	LTE_EVENT_ERROR	REBOOT_MODEM
30	test_stray_ok_in_run	OK	RUN

6 Key ideas

Key idea

1. **One command in flight.** Replies are routed by `s_current_cmd`, so an `OK` can never be credited to the wrong command.
2. **A failure is an error reply or a timeout.** Both add 1 to the command's retry counter.
3. **A success resets the counter.** Only consecutive failures count.
4. **Retry, then reboot.** Below 3 failures the command is resent; at 3 the modem is rebooted, which also clears every counter.
5. **Boot has its own retry.** Each boot command is sent up to 15 times, 2 s apart, and the counter restarts for every step.
6. **URCs are handled first and belong to no command.** `+CEREG` can announce a lost network at any time.
7. **Ticks make silence recoverable.** Recovery also runs on the tick, so a modem that sends nothing still gets recovered.

6.1 Interview sentences

Each sentence is something you can say in your own words.

- "I simulate the modem by injecting scripted replies and moving a fake clock, then assert the state machine ends in the right state for every success, error, timeout, and retry-limit case."
- "Both an error reply and a timeout count as a failure; only a success or a modem reboot clears the counter."
- "Each command is sent at most three times; the third failure escalates to a modem reboot, so the device cannot get stuck retrying forever."
- "During boot I resend the same command every two seconds, up to fifteen times, then reboot the modem."
- "Registration reports are handled before command replies, because they can arrive at any time and belong to no command."
- "I use `deliver_line` when I want to see the state right after a failure and `step_sm` when I want the next command to be sent."

7 Limits of the suite

These are observations from reading the code, not faults. Change them one at a time, if at all.

7.1 What the tests do not check

- **Commands sent.** The tests check *states*, not which AT command was sent. A wrong command with the right state would still pass. Capturing what the driver sends would close this gap.
- **Two states accepted.** Three tests accept either `ERROR` or `REATTACH`:

  - `test_attach_error_retry`
  - `test_cereg_network_lost`
  - `test_timeout_attach`

  With the current code the result is `REATTACH` in all three, so they could be made exact.

- **Hard-coded constants.** The values 3, 2000, 15, 8000, 10000, and 30001 are copied into the tests. If a constant changes in `lte_sm.c`, the matching test must change too.

7.2 Behaviour with no test

- A late reply after a timeout (a known limitation noted in the source comments).
- `+CEREG` values 2, 4, and 5, and unparsable `+CEREG` lines.
- The answer `+CGATT: 0` treated as a failed attach, and `+CMS ERROR`.
- An `ERROR` reply to `AT` or `ATE0` during boot (the timeout resends).
- An external `LTE_EVENT_TIMEOUT` event (only `LTE_EVENT_ERROR` is tested).
- The `LTE_SM_HONOR_TEST_FLAGS == 1` build option.
- Several commands failing in sequence, where the counters of different commands interact.

7.3 Behaviour the code does not have yet

The state machine has no check of the connection while in `RUN`, no pause between reboots, and no separate state for configuration errors that retrying cannot fix. The tests correctly say nothing about these, because the code does not implement them.

7.4 Candidate additions

If you extend the suite, add one test at a time and make each pass before starting the next:

1. Tighten the three two-state tests to expect `REATTACH`.
2. Add `+CEREG: 5` and `+CEREG: 2` cases (no state change).
3. Add an `ERROR` reply to `AT` during boot (resent, no state change).
4. Record what the driver sends, and assert the command text in the PDP and attach tests.

A Glossary

AT command	A text command sent to the modem over the serial line, such as `AT+CGATT=1`.
Final result	The line that ends a command: `OK`, `ERROR`, `+CME ERROR: n`.
Information line	A line before the final result that carries data, such as `+CPIN: READY`.
URC	Unsolicited result code: a line the modem sends by itself, such as `+CEREG: 1`.
In flight	A command that was sent and has no final result yet.
Attach	Joining the LTE network (`AT+CGATT=1`).
Registration	The modem's network status, reported by `+CEREG`.
PDP context	The data session that gives the modem an IP address (`AT+CGACT=1,1`).
Tick	A periodic call with no event: `lte_sm_handle_event(NULL)`.
Recovery state	A state that only exists to repair a failure: `REATTACH`, `REPDP`, `REOPEN_SOCKET`, `MQTT_RECONNECT`, `MQTT_RESUBSCRIBE`, `REBOOT_MODEM`.
Fake clock	The test clock that only advances when `lte_hw_delay_ms()` is called.